

Programmazione e calcolo scientifico

Esercitazione 8 - Implementazione dei grafi

5 maggio 2026

1 Obiettivo dell'esercitazione

Si implementi una struttura dati in grado di rappresentare un grafo non diretto. Prima di iniziare, ci si documenti sommariamente sui seguenti contenitori della STL, che verranno a breve ripresi a lezione:

- `std::list` (<https://cppreference.com/cpp/container/list>)
- `std::set` (<https://cppreference.com/cpp/container/set>)
- `std::map` (<https://cppreference.com/cpp/container/map>)
- `std::unordered_map` (https://cppreference.com/cpp/container/unordered_map)

Nella propria implementazione della struttura dati grafo si utilizzino opportunamente i contenitori della STL menzionati sopra.

2 Specifiche di quanto dev'essere implementato

Si implementi una classe `undirected_edge` che rappresenta un arco in un grafo. La classe deve avere un costruttore che permette di specificare i due nodi connessi dall'arco, inoltre devono esserci due metodi `from()` e `to()` che restituiscono i due nodi. Si implementi anche :

- `operator<` per `undirected_edge`, affinché gli archi possano essere ordinabili,
- `operator==` per `undirected_edge`,
- `operator<<` per `undirected_edge`.

Si faccia in modo che `undirected_edge` garantisca sempre che `from` è minore di `to`.

Si passi quindi all'implementazione di una classe `undirected_graph`. Essa deve prevedere:

- Un costruttore di default
- Un costruttore di copia
- Un metodo `neighbours()` che, dato un nodo, restituisce i suoi vicini,

- Un metodo `add_edge()` che permetta di aggiungere un arco al grafo,
- Un metodo `all_edges()` che restituisce tutti gli archi,
- Un metodo `all_nodes()` che restituisce tutti i nodi,
- Un metodo `edge_number()` che, dato un arco, ne restituisce la sua numerazione all'interno del grafo,
- Un metodo `edge.at()` che, dato un numero d'arco, restituisce il corrispondente oggetto arco all'interno del grafo,
- L'operatore `operator-`, che permette di calcolare la differenza tra due grafi: dati G e G' , la differenza $G - G'$ è data dagli archi presenti in G e non presenti in G' .

Si noti che l'operazione `add_node()` non è stata menzionata. Si propongano e si scrivano opportuni test per la struttura dati implementata.

3 Modalità di svolgimento e consegna

Deve essere consegnato un programma che implementi quanto richiesto sopra più un `CMakeLists.txt` che lo compili. Il tutto deve essere posizionato in una cartella chiamata **esercitazione8** nel proprio repository.

La consegna delle esercitazioni è obbligatoria e da effettuarsi entro i termini prescritti. Il tempo assegnato per lo svolgimento di questa esercitazione è di una settimana. Termine ultimo per la consegna di questa esercitazione sono le ore 23.59 di Mercoledì 13/05/2025: fa fede il timestamp del commit taggato `consegna_es8` sul proprio repository. Eventuali giustificazioni vanno presentate a `matteo.cicuttin@polito.it` entro il termine ultimo previsto per la consegna dell'esercitazione.

Si ricordi che l'uso di strumenti di intelligenza artificiale è scoraggiato ed in ogni caso dev'essere dichiarato esplicitamente all'interno dei sorgenti consegnati.

All'esame potrebbe essere richiesto di mostrare il funzionamento del codice, pertanto ci si assicuri che il codice nel proprio repository sia funzionante.